

Core Java Lab Book

Document Revision History

Date	Revision No	Author	Summary of changes
1-Aug-2021	1.0	Pankaj Sharma	Started to create Lab activities for core java hands on
5-Aug-2021	1.1	Pankaj Sharma	As per the discussion with Divya, need to reduce the no of assignment per day as 4 for each topic.

Table of Contents

Topic.....	Page No
Document History.....	2
Table of Contents.....	3
Getting Started.....	4
Overview.....	4
Setup Checklist for core java.....	4
Instructions.....	4
Lab 1: Java Basics.....	5
Lab 2: Programming Fundamentals.....	7
Lab 3: Arrays.....	9
Lab 4: String and Parsing.....	11
Lab 5: OOPs (Class, Encapsulation, Inheritance, Polymorphism, Abstraction).....	14
Lab 6: Exception Handling.....	16
Lab 7: Collection Framework.....	19
Lab 8: Multithreading.....	24
Lab 9: Java 8, Lambda, Stream API, Functional Interfaces.....	29
Lab 10: File Handling.....	32
Lab 11: Layered Structure with JDBC(TDD, Maven, DevOps & Sonar).....	33

Getting Started

Overview

This Lab book is planned very carefully so that you can have sufficient exercises to get a very good understanding on technology. It is recommended to solve all the exercises that are given and explained by the trainer. As of now when this book is planned, we decided to have java 8. The exercises will be initially simple. As per the level of technology will increase the difficulty level of lab exercises will also be increasing.

Setup checklist for core java

Below is the hardware and software requirement to run all the lab exercises.

- Core i5 minimum recommended
- Microsoft windows 7 or higher
- HDD: Minimum 10 GB or Higher
- RAM: Minimum 4 GB
- Notepad ++
- JDK 8
- STS

Instructions

This Lab book is the companion of the Core Java Training. If you want to get the maximum of the training, make sure to complete all the Lab exercises. The activities are very carefully planned to give you the exposure of the real world development.

Total questions in this lab book

Topic	Number of Questions
Java Basics	3
Programming Fundamentals	4
Array	5
String and Parsing	4
OOPs (Class, Encapsulation, Inheritance, Polymorphism, Abstraction)	4
Exception Handling	4
Collection Framework	8
Java 8	4
File Handling	4
Layered Structure with JDBC	6

Lab 1: Java Basics

Brain Teaser-----

Q1# : You need to create a simple application. Name your main class as SampleOutput.java. When you run your main class. You should get output as below.

"Hello, Pankaj."

"Hello, Java."

Give yourself bonus point in case you do the activity as trainer is expecting you to do using Test First approach.

Q 2# : Most of the java apps will require the input from the user. That, we will be learning later as well. Right now let us focus on strategy to take input from user and display that on console.

```
Scanner scanner = new Scanner(System.in);
String str= scanner.next();
int num= scanner.nextInt();
scanner.close();

System.out.println("String is: " + str);
System.out.println("Num is: " + num);
```

Extra Task : Explore Scanner on [jdk api doc](#)

Here through Scanner we are taking the input from the standard input. Using scanner we use various methods to take specific type of input, such as String, int, float etc.

scanner.close() specify the termination of stream and says there is no more need of tacking input from console.

Below two output lines display the output of the string and number entered by user.

Your task is to take the input of the below requirement.

Scenario Based-----

Q3: Assume we are working on Product Management app, where we need to enter the product based details in the system. Such as productName, MRP, Selling price, and discount provided. We are assuming the output as below.

Output:

Sr #	Product Name	MRP	Selling Price	Discount
1	Headphone	500	400	20

Lab 2: Programming Fundamentals

Brain Teaser-----

Q1 : Create a method to find the sum of the cubes of the digits of an n digit number

Q2: Write a program that should ask a number from user. You need to display all the prime numbers till that number. For exam : 25

Intermediate Mandatory -----

Q3: Create a class NaturalNumberSumDemo, Here you need to calculate the sum of first n natural numbers which are divisible by 3 or 5. Follow below table:

ClassName	NaturalNumberSumDemo
Helper Method Name	calculateSum
Method Description	To find the sum first n natural number divisible by 3 and 5
Argument	int number
Return type	Int
Logic	Calculate the sum of first n natural numbers divisible by 3 and 5

Scenario Based -----

Q4: Imagine that you are working on a library management app. In this app, members can get the books issued. We have a data of books which are issued by members. Your job is to find out the average number of books that are issued in a month by library members.

Follow the below instructions to attempt this exercise.

1. Create a Person class
2. Keep id, name, books_issued fields in Person class.
3. Create a parameterized constructor in Person class, and you must have getters for all fields.
4. Create a LibraryDemo class with main method
5. Use below table for further reference.
6. Make sure to use modular approach.

Persons data	<pre>Person[] persons=new Person[]{\n new Person(101,"pankaj",3),\n new Person(102,"vikas",2),\n new Person(103,"vineet",3),\n new Person(104,"akash",1),\n new Person(105,"shalini",0),\n new Person(106,"shikha",1),\n new Person(108,"prateek",3)\n};</pre>
Get only books_issued number	<pre>private int[] getBooksIssued(Person[] persons){\n //TODO: Write your logic here</pre>

	}
Find the average books issued by all members	private int findAvgBookIssued(int[] books_issued){ //TODO: Write your logic here }

Lab 3: Arrays

Brain Teaser-----

Q1: Ten numbers are entered from the keyboard into an array. The number to be searched is entered through the keyboard by the user. Write a function which should get the array as input and a number to be searched. This function should return the frequency of a searched number in provided list of numbers.

Q2: Write a program to copy the contents of one array into another array in the reverse order, without using any inbuilt method.

Intermediate-----

Q3: Write a function `reverself()`, that reverse a String. Use a for loop that swaps the first and last characters, then second and next-to-last characters and so on. The string should be passed to `reverself()` method as an argument and it should return the reverse string. Write a program to exercise `reverself()`. The program should get the string from the user. Call `reverself()` and print the output. Check the program with the following String "Able was I era I saw elba.".

Scenario Based-----

Q4: Create a 5 story car parking system simulation, where ground floor will be having the capacity to park 5 cars, 2nd story will have the capacity to park 4 cars, 3rd story will have the capacity to park 3 cars, 4th story will have the capacity to park 2 cars and finally 5th story will have the capacity to park 1 car.

ClassName	Car
Fields	regNo, ownerName, tokenNo
Methods	Generate all getter and setter and parameterized constructor
ClassName	CarParking
Methods	showAvailability() : this will show the available space
	parkCar(Car car): this will park the car on available place
	getCar(Car car): this will get the car for owner based on provided tokenNumber. And make the same place available for next parking
ClassName	CarParkingDemo
Method Name	main(String[] args): this will check all the operations of the CarParking.

Note: You must have a menu driven program for this last exercise.

Q5: You need to create your own `ArrayUtil` class that will be providing you multiple utility methods that will be helping you out to perform operations on Any array.

Note: Try to attempt this exercise using TDD. (It is optional)

Class Name	ArrayUtil
Method	searchUsingBinarySearch(int[] arr, int key): this will search the specified int array for the specified key using binary search algorithm.
	searchUsingBinarySearch(float[] arr, float key): this will search the specified float array for the specified key using binary search algorithm.
	searchUsingBinarySearch(Object[] arr, object key): this will search the specified object array for the specified key using binary search algorithm.
	static int[] arrayCopy(int[] original, int newLength): Copies the specified array, newLength is the increased length of the new array.
	static void sort(int[] arr, int fromIndex, int toIndex): Sorts the specified range of specified array of objects into ascending order according to the natural ordering of its elements.
Important Note	You need to work on float and object variation for sort and arrayCopy method as well.
Restrictions	You are not suppose to use the inbuilt methods for this exercise.
ClassName	CheckArrayUtil
Method name	main(String[] args):This method is used to test the ArrayUtil class operations.

Lab 4: String and Parsing

Brain Teaser-----

Q1: Write a program to get the firstname, lastname and year of birth as input and return the generated password. Follow the below table for more understanding.

Class Name	StringUtil
Method	public String generatePassword(String firstName,String lastName, String yob): This should return the generated password.
Sample Input	PanKaj SHaRma 1983
Output	ankhar83
Note	User may give input in mixed case, your generated output must be in lowercase only
Class Name	StringUtilTest
Method	main(String[] args): For testing the StringUtil method

Intermediate-----

Q2: Create a method in the StringUtil class as, getCommand(). This method will receive on url and will return the command from this url. Follow below table for more understanding.

Class Name	StringUtil
Method	public String getCommand(String url): This method will receive the url and return the command associated with that url.
Sample Input	www.sample.com/admin/createProject.html
Output	createProject
Note	Check with multiple and different type of urls
Class Name	StringUtilTest
Method	main(String[] args): For testing the StringUtil method

Scenario Based-----

Q3: You need to work on the **StringExercises** class, that will have some set of static methods. You need to implement these methods as per the description. For testing different operations of the StringExercises class. Create a main class and accordingly call the methods with appropriate parameters.

Class Name	StringExercises
<pre>public static int findMarc(String string) { return -2; }</pre>	Searches for "Marc" in a string. string is a non null String return the index of the first occurrence of the "Marc" in the string. Otherwise -1 if not found
<pre>public static int findSubstring(String string, String substring) { return -2; }</pre>	Searches for substring within a string string is a non-null string substring a non-null string return the index of the first occurrence of the substring within the string, or -1 if not found
<pre>public static boolean contains(String string, String substring) { return false; }</pre>	Returns true if and only if the string contains the substring. string is a non-null string substring is a non-null string return true if and only if the string contains the substring
<pre>public static String[] splitIntoWords(String string) { return null; }</pre>	Splits a string into words, using whitespace to delimit the words. you can use the String.split method as follows to get an array of the words: String[] words = s.split("\\s+"); string a non-null string return an array representing the words in the string.
<pre>public static String firstFour(String string) { return null; }</pre>	Returns the substring representing the first four characters of the string. string a non-null string of length>=4 return the substring representing the first four characters of the string.
<pre>public static String firstN(String string, int n) { return null; }</pre>	Returns the substring representing the first n characters of the string. string a non-null string of length>=n n an integer>=0 return the substring representing the first n characters of the string
<pre>public static String lastFour(String string) { return null; }</pre>	Returns the substring representing the last four characters of the string. string a non-null string of the length >=4 return the substring representing the last four characters of the string
<pre>public static String lastN(String string, int n) { return null; }</pre>	Returns the substring representing the last n characters of the string. string a non-null string of length>=n n an integer>=0 the substring representing the last n characters of the string.

Q4: Create a class named as **MatchPatternUtil**, and you will have to create few methods in this. Follow the below table for method description. For testing the MatchPatternUtil class, create a main class and test functionality with appropriate input.

Class Name	MatchPatternUtil
public static int[] matchabSequence(String string){ return null; }	Return the array of int, which holds the index number of matched pattern. string is non-null string input: abaaaba output: 0 4
public static int[] findDigitLocation(String string){ return null; }	Return the array of int, which holds the index number of matched pattern. string is non-null string input: a12c3e456f output: 1 2 4 6 7 8
public static int[] findWhiteSpaces(String string){ return null; }	Return the array of int, which holds the index number of matched pattern. string is non-null string input: a 1 56 _Z output: 1 3 6
public static int[] findPattern(String string){ return null; }	Return the array of int, which holds the index number of matched pattern. string is non-null string input: 12 0x 0x12 0Xf 0Xg output: 6 11 What to do : Find a set of characters in which the first character is a "0", the second character is either an "x" of an "X", and the third character is either a digit from "0" to "9", a letter from "a" to "f" or an upper case letter "A" to "F."

Lab 5: OOPs

Brain Teaser-----

Q1: Work on the below requirement.

1. Make a parent class Person.
 - a. Make Employee, UnEmployed and SelfEmployed as the subclasses. Make calculateSalary method as a base method in Person class.
 - b. Override this calculation method in all the other classes (Employee, Unemployed, and SelfEmployed)
 - c. Make parameterized constructor in individual derived subclasses which takes calculation parameters (like tax %, Salary, etc) as parameters and does the salary calculations.
 - d. Make another class CalculateSalary with overloaded parameterized constructors. These parameterized constructors take empType and other parameters as tax%, salary etc. and calculates the salary.
2. Make use of access Specifies in the above program:
 - a. Make the calculateArea method private in the parent class.
 - b. Make the overridden methods public in the subclasses.

Q2: Create a class that imitates part of the functionality of the basic data type int. Call the class Int (note different capitalization). The only data in this class is an int variable. Include member functions to initialize an Int to 0, to initialize it to an int value, to display it (it looks just like an int), and to add two Int values.

Write a program that exercises this class by creating one uninitialized and two initialized Int values, adding the two initialized values and placing the response in the uninitilized value, and then displaying this result.

Follow below table for more understanding.

Class Name	Public class Int
Field	private int sum
Must have	Default constructor, that will initialize the sum with default value as 0
	Getter for sum field
public void addNumber(int i, int j){}	Will add the value of two ints and keep it in sum
ClassName	public class CheckInt
Method	main(String[] args): to test Int class

Intermediate-----

Q3: Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay Rs 50 toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars

that have gone by, and of the total amount of money collected. Model this tollbooth with a class called TollBooth. The two data items are a type int to hold the total number of cars, and a type double to hold the total amount of money collected. A constructor initializes both of these to 0. A member function called payingCar() increments the car total and adds Rs 50 to the cash total. Another function called noPayCar(), increments the car total but adds nothing to the cash total. Finally, a member function called display() displays the two totals. Make appropriate member functions.

Create a main class to test this app. This program should allow the user to push one key to count a paying car, and another to count a nonpaying car. Pushing the Esc key should cause the program to print out the total cars and total cash and then exit.

Scenario Based-----

Q4: You need to work on Doctor Information System. Below is the problem definition and business requirement.

Problem Definition

You need to store the information of Doctors, which is going to hold information like, name, speciality, and Patient. Patient will have information like name, problem. In this application doctors will be of below specialities. Neurologist, Dermatologist, Dentist, Orthopedist. If patient inquire about Doctor's speciality which is not available in record, specific exception must be reported. Data should be stored in array as local repository. Use the standard practice of development, your code must be properly structured and documented.

Business Requirement

- Application must have proper table structure with proper relationships.
- There should not be a prefilled data local repository.
- Perform all the CRUD operations on arrays
- You must use the concepts of oops properly at appropriate situations
- Modular approach must be used in application development
- Create a menu driven program
- You must have main menu that must have operations related to Doctor and Patients

Lab 6: Exception Handling

Brain Teaser-----

Q1: I am providing you the skeleton code. Write it as it is in your code editor. And follow the below instruction.

Add a **readList** method to **ListOfNumbers.java** (in a .java source file). This method should read in int values from a file, print each value, and append them to the end of the vector. You should catch all appropriate errors. You will also need a text file containing numbers to read in.

```
import java.io.*;
import java.util.Vector;

public class ListOfNumbers {
    private Vector vector;
    private static final int size = 10;

    public ListOfNumbers () {
        vector = new Vector(size);
        for (int i = 0; i < size; i++)
            vector.addElement(new Integer(i));
    }

    public void writeList() {
        PrintStream out = null;

        try {
            System.out.println("Entering try statement");
            out = new PrintStream(new FileOutputStream("OutFile.txt"));

            for (int i = 0; i < size; i++)
                out.println("Value at: " + i + " = " + vector.elementAt(i));
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println("Caught ArrayIndexOutOfBoundsException: " +
                               e.getMessage());
        } catch (IOException e) {
            System.err.println("Caught IOException: " + e.getMessage());
        } finally {
            if (out != null) {
                System.out.println("Closing PrintStream");
                out.close();
            } else {
                System.out.println("PrintStream not open");
            }
        }
    }
}
```

Extra Reading: You must have some basic understanding on File IO and vector data structure.

Q2: Write a program for the StringCalculator. This program will take the comma separated numbers and print the sum of the numbers as output. You need to make sure that the numbers that are input in String form must be numeric, not the string. You need to handle the proper exception and show the proper error message to user.

Follow below table for more understanding.

Class Name	StringCalculator
Method	public int calculate(String input){return 0;}
Description	Input will be comma separated numbers. And output will be the sum of all the numbers that are provided in input. For example Case 1: "10,20" Output : 30 Case 2: "10" Output: 10 Case 3: "" Output: 0 Case4:"One, 2" Output: Invalid Input

Intermediate-----

Q3: Create a BankService, which will have withdraw method. This method will receive two input as account number and amount to be withdrawn. When this method is called, it should prompt that it may throw InvalidAccountNumberException and InsufficientAmountException. Programmer will have to handle these exception.

Assume that you will have some initial balance. So incase if amount that is withdrawn exceeds the balance then InsufficientBalanceException must be reported.

If account number is not valid then InvalidAccountNumberException must be reported.

Create a separate class to test the functionality of the BankService.

Scenario Based-----

Q4: Refer the Exercise 3 from the OOPs Lab (Doctor Information System). There are multiple places where different type of issues can come. You need to Create appropriate Exceptions and handle them properly in the application. So this is the addition of your OOPs Lab, Question 3.

Follow below table for the different type of issues and the probable Exception that you need to manage.

When saving Doctor detail. Check before saving that doctor object should not be null. Otherwise appropriate Exception should be reported	Look for the NullPointerException
When accessing or trying to save the object out of array index.	Look for the ArrayIndexOutOfBoundsException
When accessing the doctor by id	DoctorNotFoundException: Need to create a custom Exception. Create it as RuntimeException. And understand the importance of RuntimeException
When accessing the patient by id	PatientNotFoundException: make it as checked exception.
Note:	You must handle the same exceptions for Patients as well. While saving patient detail.
Handle situation, when patient looks for doctor as per the speciality.	SpecialityNotFoundException: Create a custom exception, that must be handled, so that if patient look for any other specialty which is not supported.

Lab 7: Collection Framework

Brain Teaser-----

Q1: For your own exercise, do the following.

- Create your own eclipse project as MyHashSet
- Create your own HashSet object with initial capacity of 5
- Add the following objects to the newly created HashSet object
 - 2 String Objects
 - 3 MyOwnClass Object (you will have to first create the MyOwnClass.java class)
 - 3 Integer Objects
- Display the Hash set object
- Try to add same object once again and see the behaviour.
- Observe the behaviour of HashSet working.

Q2: You have a list of names, your job is to find out which word or words are duplicate, and show the list of distinct names. Follow the below code and complete the code.

```
public class Set_HashSetFindDuplicate {  
  
    public static void main(String[] args) {  
        // set up test data  
        String names[]={  
            new String("pankaj"),  
            new String("rajesh"),  
            new String("suresh"),  
            new String("pankaj")  
        };  
    }  
}
```

-----Initial list-----

pankaj, rajesh, suresh, pankaj

Duplicate name detected :pankaj

3 distinct words detected : list : [suresh, pankaj, rajesh]

Q3: Modify the above app, and name it as Set_HashSetFindDuplicate2 .In this app you need to show the list of unique names and show which names are duplicate. Find the below code and add your logic.

```
public class Set_HashSetFindDuplicate2 {  
  
    public static void main(String[] args) {  
        // set up test data  
        String names[]={  
            new String("pankaj"),  
            new String("rajesh"),  
            new String("suresh"),  
            new String("pankaj"),  
            new String("suresh"),  
            new String("aman")  
        };  
    }  
}
```

Your output should be as below.

-----Initial list-----

pankaj, rajesh, suresh, pankaj, suresh, aman

unique names : [aman, rajesh]

duplicate names : [suresh, pankaj]

Intermediate-----

Q4: Create an application to add document detail as (id, title, description). This detail should be added permanently in file system. Your application should read all the documents from the docs.txt file and show separately how many documents are duplicated for how many number of times, and show the list of distinct documents.

Q5: Create two TreeSet objects and put them in two set references, one object will hold the string type value and other object will hold the integer type values. Observe the behaviour of adding values in TreeSet. Come up with solution how it is happening?

Map Assignments-----

Q6: You need to find out the frequency of the word repeated in an array of names. Attempt it using Map. You can use the below skeleton for your reference.

```
package map;
import java.util.HashMap;
import java.util.Map;
public class Map_HashMap {
    private static final Integer ONE=new Integer(1);
    public static void main(String[] args) {
        // Set up testing data
        String name[] = {
            new String("pankaj"),
            new String("rajesh"),
            new String("pankaj"),
            new String("deepak"),
            new String("pankaj")
        };
    }
}
```

Output should be as below

frequency of :pankaj is : 1

frequency of :rajesh is : 1

frequency of :pankaj is : 2

frequency of :deepak is : 1

frequency of :pankaj is : 3

3 distinct word detected

Display of HashMap object : {deepak=1, pankaj=3, rajesh=1}

Optional-----

Q7: Creating your own collection.

In this exercise you need to create your own ArrayList. You are not suppose to use the in build methods of the ArrayList. For the specified operation you need to create your own logic. But it should work same as ArrayList.

For this exercise some setup activity is as below.

Create Document with below details

- name
- purpose
- validity

- showDocumentDetail

- You may have different type of documents.like Personal, Official, Confidential etc.
- Create a list of documents and display the report of each type of documents. (User your List Interface and Your ArrayList Class)

Limitations

- list should be used from your own collection api, not from java.util.
- list should accept only Documents, other types should not be allowed.

Perform below operations

- Adding
- Searching
- Removing
- Replacing
- Find total documents

Scenario Based-----

Q8: Create ListExercises class with below methods. Method description is provided. You need to create a main class to test all the functionalities of the ListExercises.

Class Name	ListExercises
<pre>public static int countCharacters(List<String> list) { return 0; }</pre>	Counts the number of characters in total across all strings in the supplied list. In other words, the sum of the lengths of the all the strings. list is a non null list of string. return the number of characters.
<pre>public static List<String> split(String string) { return null; }</pre>	Splits a string into words and returns a list of the words. If the string is empty, split returns a list containing an empty string. string a non-null string of zero or more words return a list of words
<pre>public static List<String> uppercased(List<String> list) { return null; }</pre>	Returns a copy of the list of strings where each string has been uppercased (as by String.toUpperCase).The original string is unchanged. List is a non null list of string return a list of uppercased strings
<pre>public static boolean allCapitalizedWords(List<String> list) { return false; }</pre>	Returns true if and only if each string in the supplied list of strings starts with an uppercase letter. If the list is empty, returns false. list is a non null list of string return true if each string starts with an uppercase letter
<pre>public static List<String> filterContaining(List<String> list, char c) { return null; }</pre>	Returns a list of strings selected from a supplied list, which contain the character c. The returned list is in the same order as the original list, but it omits all strings that do not contain c. The original list is unmodified. list is a non null list of string c the character to filter on return a list of strings containing the character c, selected from list.
<pre>public static void insertInOrder(String string, List<String> list) { } }</pre>	Inserts a string into a sorted list of strings, maintaining the sorted property of the list. string is the string to insert list is a non null sorted list of strings

Class Name	ListExercisesTest
public static void main(String [] args){ //TODO : Your testing code goes here }	Use main method to test your ListExercises class's methods.

Lab 8: Multithreading

Brain Teaser-----

Q1: Write a Java program which handles Push operation and Pop operation on stack concurrently.

Q2: Write a Java program which first generates a set of random numbers and then determines negative, positive even, positive odd numbers concurrently.

Intermediate-----

Q3: Create two threads, one thread to display all even numbers between 1 & 20, another to display odd numbers between 1 & 20. Note: Display all even numbers followed by odd numbers Hint: use join

Scenario Based-----

Q4: I am providing the below code snippet that you need to write in a simple java project. So follow below steps and then attempt to solve the problem statement.

Steps

Step1: Create a core java project with the name as **AdvanceThreadDemo**

Step2: Create 4 classes separately in src directory. Code is as below.

Classes

1. BankAccount
2. Producer extends Thread
3. Consumer extends Thread
4. Bank ----- with main method

```

2 public class BankAccount {
3     private double balance;
4     public BankAccount(double balance) {
5         this.balance = balance;
6     }
7     public BankAccount() {
8         this(0);
9     }
10    public double getBalance() {
11        return balance;
12    }
13    public void deposit(double amount) {
14        double temp = balance;
15        temp = temp + amount;
16        try {
17            Thread.sleep(300); // simulate production time
18        } catch (InterruptedException ie) {
19            System.err.println(ie.getMessage());
20        }
21        System.out.println("after deposit balance = $" + temp);
22        balance = temp;
23    }
24    public void withdraw(double amount) {
25        if (balance < amount) {
26            System.out.println("Insufficient funds!");
27            return;
28        }
29        double temp = balance;
30        temp = temp - amount;
31        try {
32            Thread.sleep(200); // simulate consumption time
33        } catch (InterruptedException ie) {
34
35            System.err.println(ie.getMessage());
36        }
37        System.out.println("after withdrawl balance = $" + temp);
38        balance = temp;
39    }
40 }

```

```

1 //A typical producer deposits $10 x 5 = $50:
2 public class Producer extends Thread{
3     private BankAccount bankAccount;
4     public Producer(BankAccount bankAccount) {
5         super();
6         this.bankAccount = bankAccount;
7     }
8     @Override
9     public void run() {
10        for (int i = 0; i<5; i++) {
11            bankAccount.deposit(10);
12        }
13    }
14 }
15

```

```

1 //A typical consumer withdraws $10 x 5 = -$50:
2 public class Consumer extends Thread{
3     private BankAccount bankAccount;
4     public Consumer(BankAccount bankAccount) {
5         super();
6         this.bankAccount = bankAccount;
7     };
8
9     @Override
10    public void run() {
11        for (int i = 0; i < 5; i++) {
12            bankAccount.withdraw(10);
13        }
14    }
15 }

```

```

1 //The master thread is the bank. It creates an account with an initial balance of $100. It creates 4 account holders, 2 producers and 2 consumers:
2 public class Bank {
3     public static void main(String[] args) {
4         BankAccount bankAccount = new BankAccount(100);
5         int accountHolder = 4;
6         Thread[] accounts = new Thread[accountHolder];
7
8         // Creating two producers and 2 consumers
9         for (int i = 0; i < accountHolder; i++) {
10            if (i % 2 == 0) {
11                accounts[i] = new Producer(bankAccount);
12            } else {
13                accounts[i] = new Consumer(bankAccount);
14            }
15        }
16        // Starting the operation of both producers and consumers
17        for (int i = 0; i < accountHolder; i++) {
18            accounts[i].start();
19        }
20        // Use of join
21        for (int i = 0; i < accountHolder; i++) {
22            try {
23                accounts[i].join();
24            } catch (InterruptedException e) {
25                System.out.println(e.getMessage());
26            } finally {
27                System.out.println("account " + i + " has died");
28            }
29        }
30        System.out.print("Closing balance = ");
31        System.out.println("$" + bankAccount.getBalance());
32    }
33 }
34

```

Now run the main class and observe the output.

terminated / Bank Java Application] C:\spring-to

```
after withdrawl balance = $90.0
after withdrawl balance = $90.0
after deposit balance = $110.0
after deposit balance = $110.0
after withdrawl balance = $80.0
after withdrawl balance = $80.0
after deposit balance = $120.0
after deposit balance = $120.0
after withdrawl balance = $70.0
after withdrawl balance = $70.0
after withdrawl balance = $60.0
after withdrawl balance = $60.0
after deposit balance = $130.0
after deposit balance = $130.0
after withdrawl balance = $50.0
after withdrawl balance = $50.0
after deposit balance = $140.0
after deposit balance = $140.0
after deposit balance = $150.0
account 0 has died
account 1 has died
after deposit balance = $150.0
account 2 has died
account 3 has died
Closing balance = $150.0
```

Problem Statement:

Notice that the “insufficient funds” message never appeared. That means both consumers successfully withdrew \$10 five times each for a total of -\$100. Similarly, both producers successfully deposited \$10 five times each for a total of \$100. The initial balance was \$100, so the closing balance should have been:

$$\text{balance} = \$100 + \$100 - \$100 = \$100$$

somehow, the bank lost \$50.

It seems pretty clear what went wrong. Look at the first three lines of output:

```
after withdrawl balance = $90.0
after withdrawl balance = $90.0
after deposit balance = $110.0
```

Apparently, when the first producer was in the middle of making a deposit, he was interrupted by a consumer who quickly withdrew \$10, leaving the balance at \$90. Unfortunately, the last time the producer checked, the balance was \$100, so after adding an additional \$10, the producer now believes the balance is \$110. In other words, the bank lost track of the \$10 withdrawn by the consumer.

Make the appropriate changes in your app so that you must get the output as below.

```
after deposit balance = $110.0
after deposit balance = $120.0
after deposit balance = $130.0
after withdrawl balance = $120.0
after withdrawl balance = $110.0
after withdrawl balance = $100.0
after withdrawl balance = $90.0
after withdrawl balance = $80.0
after withdrawl balance = $70.0
after withdrawl balance = $60.0
after deposit balance = $70.0
after deposit balance = $80.0
after deposit balance = $90.0
after deposit balance = $100.0
after deposit balance = $110.0
after withdrawl balance = $100.0
after withdrawl balance = $90.0
after withdrawl balance = $80.0
after deposit balance = $90.0
after deposit balance = $100.0
account 0 has died
account 1 has died
account 2 has died
account 3 has died
Closing balance = $100.0
```

Note : the sequence of deposit or withdraw can be changed. But finally the closing balance must be \$100. Because same number of withdraw and deposits are happening with same amount in the system, so ideally closing balance must be initial balance only i.e. \$100;

Lab 9: Java 8, Lambda, Stream API, Functional Interfaces

Brain Teaser-----

Basic Setup:

To perform the operations on the Lambda, Stream API and Functional Interfaces in the below exercises you need some basic setup. So create the setup as instructed and attempt all the questions of that.

- Create a Project with the name as **Java8Demos**.
- Create two packages with the name as **commons** and **handson**.
- In the **commons** package create one **Enum** and one **class** as below.

```
public enum Gender {  
    MALE, FEMALE  
}  
  
public class Person {  
    private String name;  
    private int age;  
    private Gender gender;  
    public Person(String name, int age, Gender gender) {  
        super();  
        this.name = name;  
        this.age = age;  
        this.gender = gender;  
    }  
    public String getName() {return name;}  
    public void setName(String name) {this.name = name;}  
    public int getAge() {return age;}  
    public void setAge(int age) {this.age = age;}  
    public Gender getGender() {return gender;}  
    public void setGender(Gender gender) {this.gender = gender;}  
    @Override  
    public String toString() {  
        return "Person [name=" + name + ", age=" + age + ", gender=" + gender + "];"  
    }  
}
```

In the hands on package create below class.

```
public class Practice01 {  
  
    public static List<Person> createPeople(){  
        return Arrays.asList(  
            new Person("pankaj", 38, Gender.MALE),  
            new Person("Manvi", 6, Gender.FEMALE),  
            new Person("Aman", 34, Gender.MALE),  
            new Person("Bindu", 23, Gender.FEMALE),  
            new Person("Kabir", 45, Gender.MALE),  
            new Person("Monika", 45, Gender.FEMALE),  
            new Person("Monika", 35, Gender.FEMALE),  
            new Person("Vijay", 34, Gender.MALE),  
            new Person("Priyanka", 35, Gender.FEMALE));  
    }  
  
    public static void main(String[] args) {  
        List<Person> people = createPeople();  
    }  
}
```

Now use this Practice01 for all the questions as the starting point of the application.

Q1: Get the name of all females in upper case who are in age more than 6. Attempt this without java8 concepts. (Note : This is the imperative way of working).

Q2: Get the name of all females in upper case who are in age more than 6. Attempt this by using filter, map and collect method of stream api. Use Predicate and Function functional interface as the argument of filter and map methods. (Note : This is the declarative way of working.)

Intermediate-----

Q3: Get the name of all females in upper case who are in age more than 6. Attempt this by using filter, map and collect method of stream api, now in this instead of Predicate and Function functional interfaces, use Lambda expression. (More concise way of writing code using Java8)

Scenario Based-----

Q4: You need to create a program which will check the extension of the files available in specified folder, and display the name of the filtered files.

Use the below approach for this exercise.

Approach# :With Lambda Function approach

Step1: Create **FileFilterDemoUsingLambda** Class with main method

Step2: Create a Reference of FileFilter.

Step3: Reference of FileFilter should hold the lambda function that will hold the logic for FileFilter

Step4: Using File IO display the filtered files from Files folder

Step5: You must be creating the files.txt file in your app. Which must be holding the name of the files with some extension.

Lab 10: File Handling

Brain Teaser-----

Q1: Write a main program that copies a given text file into another file using `SimpleReader` to read the input file and `SimpleWriter` to write the output file. The names of the input text file to be copied and of the destination file where the copy is to be saved are provided as command-line arguments. Assume that appropriate arguments will be provided and no error checking is necessary. The command-line arguments are accessible by your main program through the `String[] args` array parameter to the main method.

Q2: Copy and modify the previous main program so that it also includes appropriate error checks on the command line arguments, e.g., missing arguments, missing input file, non-readable input file, etc. (The `java.io.File` class provides helpful methods to check some of the possible errors.)

Intermediate-----

Q3: Write a program that will count the number of lines in each file that is specified on the command line. Assume that the files are text files. Note that multiple files can be specified, as in `"java LineCounts file1.txt file2.txt file3.txt"`. Write each file name, along with the number of lines in that file, to standard output. If an error occurs while trying to read from one of the files, you should print an error message for that file, but you should still process all the remaining files.

Scenario Based-----

Q4: Write a phone directory program that keeps its list of names and phone numbers in a file. The user of the program should be able to look up a name in the directory to find the associated phone number. The user should also be able to make changes to the data in the directory. Every time the program starts up, it should read the data from the file. Before the program terminates, if the data has been changed while the program was running, the file should be re-written with the new data. Designing a user interface for the program is part of the exercise. Create a menu driven program for this exercise.

Lab 11: Layered Structure with JDBC(TDD, Maven, DevOps & Sonar)

This lab exercise includes the whole knowledge that you got during Core Java Training. In this Lab exercise you need to complete 6 tasks.

1. Creating Maven based project for supporting TDD and Java Source code.
2. Creating layered structure for modular development.
3. Creating documented code.
4. Implementing JDBC
5. Writing Test cases for Utilities, DAO and Service layer. Work using Test First Approach.
6. Running the Sonar report for code coverage and pushing code on git repository.

You will be working on the below case study.

Problem Definition

You need to store the information of Laptop, which is going to hold information like: id,type, specification. Type could be LG, Sony, HP,DELL.Specification could have details like operating system installed, processor, hdd, ram etc. If type is other thenLG,Sony, HP, Dell then appropriate exception must be reported. Data should be stored in data base. Use the standard practice of development, your code must be properly structured and documented.

Business Requirement

- You must have initial GIT repository, where you need to incrementally push the code as per feature branch from STS.
- You need to work on Test First Approach. There must be one test suite, which will run all the test cases in application.
- Use the OOPs concepts wherever it is required.
- Application must have proper table structure with proper relationships.
- There should not be prefilled data in tables.
- Perform all the CRUD operations.
- Your application must be menu driven app.
- Follow the complete package structure and make application modular as per DAO, Service, Domain and Util layer.
- Design the flow of the application and send along with complete code.